



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

MLOps-Driven Framework for MECE Audience Segmentation for Retention

MLOps Tutorial Report

submitted by

Sriram Velumuri	1RV23AI117
Vijaykumar B K	1RV22AI118
Tejas Anand	1RV23AI113
Tharun Gowda P R	1RV23AI114

Artificial Intelligence and Machine Learning

2025-2026



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

DECLARATION

We, **Velumuri Sriram Kumar , Vijayakumar B K, Tejas Anand and Tharun Gowda P R**, the students of the fourth semester B.E., Department of Artificial Intelligence and Machine Learning, RV College of Engineering, Bengaluru-560059, bearing USN: **1RV23AI117, 1RV22AI118, 1RV23AI113 and 1RV23AI114** hereby declare that the EL topic titled **‘MLOps-Driven Framework for MECE Audience Segmentation for Retention’** has been carried out by us and submitted in partial fulfillment of the coursework requirements for the award of Degree in Bachelor of Engineering in **Artificial Intelligence and Machine Learning** of the **Visvesvaraya Technological University, Belagavi** during the year **2025-2026**.

Further, we declare that the content has not been submitted previously by anybody for the award of any Degree or Diploma to any other University.

We also declare that any intellectual property rights generated from this project at RVCE will be the property of RV College of Engineering, Bengaluru, and we will be among the authors.

Place: Bangalore

Date: 04/01/2026

Signature

Velumuri Sriram Kumar

Vijayakumar B

Tejas Anand

Tharun Gowda P

TABLE OF CONTENTS

Phase 1: Model Development and Foundations

Chapter 1: Introduction

1.1 Problem Statement and Objectives	5
1.2 Project Scope and Evaluation Metrics	6
1.3 Tools and Environmental Setup	6
1.4 Literature Review	7

Chapter 2: MLOps Foundations and Risk Analysis

2.1 Understanding MLOps, Risks and Requirement Analysis	8
2.2 Risk Assessment Matrix	9

Chapter 3: Data Preparation and Feature Engineering

3.1 Data Collection and Exploration Techniques	10
3.2 Data Profiling, Cleaning and Preprocessing	10
3.3 Feature Extraction and Feature Selection	10
3.4 Model Design Approaches	12

Chapter 4: Model Development and Evaluation

4.1 Model Building	12
4.2 Training, Validation and Testing	12
4.3 Evaluation Metrics and Visualization	13
4.4 Experiment Tracking	14
4.5 Interpreting Model Results and Error Analysis	16

Chapter 5: Visual Results

5.1 Images	16
------------------	----

Phase 2: Deployment, Monitoring and Governance

Chapter 6: Model Optimization and Governance

6.1 Revisiting Model Performance	22
6.2 Hyperparameter Tuning and Performance	22
6.3 Interpretability	23
6.4 Model Versioning and Registry	23
6.5 Documentation of Model Updates	23

Chapter 7: CI/CD and Deployment Automation

7.1 Overview of CI/CD for Machine Learning	24
7.2 Tools Setup: GitHub Actions	24
7.3 Automating Model Training and Testing	24
7.4 Building and Running a Deployment Pipeline	25

Chapter 8: Monitoring and Drift Management

8.1 Model Monitoring Concepts and KPIs	26
8.2 Monitoring Tools	26
8.3 Detecting Concept Drift and Data Drift	26
8.4 Model Retraining and Redeployment	27
8.5 Logging, Alerting and Dashboard Setup	27

Chapter 9: Post-Deployment Review and Ethics

9.1 Post-Deployment Performance Review	28
9.2 Continuous Improvement via Feedback Loops	28
9.3 Governance and Ethical AI Practices	28
9.4 Documentation: Model Cards and Data Cards	28
9.5 Audit and Review Checklist	29

Chapter 10: Final Evaluation and Demonstration

10.1 End-to-End MLOps Pipeline Demonstration	30
10.2 Final Model Evaluation and Results	30
10.3 Project Documentation and Reporting	31
10.4 Final Presentation and Evaluation	31

Chapter 1

INTRODUCTION

1.1 Problem Statement and Objectives

Problem Statement

In the competitive e-commerce landscape, businesses accumulate massive volumes of transactional data but often fail to fully exploit it for actionable customer insights. Traditional segmentation methods based on demographics or simple heuristics are insufficient to capture complex purchasing behaviors, spending patterns, and temporal trends. Additionally, many machine learning solutions remain limited to experimentation stages and lack scalability, reproducibility, and real-world deployment readiness. There is a clear need for an end-to-end, production-grade system that not only performs accurate customer segmentation but also integrates modern MLOps practices for automation, monitoring, and continuous improvement.

Project Objectives

The objective of this project is to develop a **robust, scalable, and production ready e-commerce segmentation model** using modern MLOps practices.

- To design and implement a robust customer segmentation framework using unsupervised machine learning techniques.
- To perform comprehensive data preprocessing and feature engineering to extract meaningful behavioral insights.
- To build, evaluate, and tune clustering models for effective customer profiling.
- To develop an end-to-end MLOps pipeline enabling reproducibility, version control, CI/CD, and scalable deployment.
- To deploy the trained model in a cloud environment and monitor its performance, latency, and data drift.
- To derive interpretable customer segments that can support business decision-making and personalized marketing strategies.

1.2 Project Scope and Evaluation

- Data ingestion: 1,000+ customer spending records aggregated monthly
- Application of unsupervised learning for customer segmentation.
- Dimensionality reduction using PCA for visualization and improved clustering performance.
- Experiment tracking, model versioning, and lifecycle management using MLflow and DVC.
- Deployment of the clustering model on Google Cloud Vertex AI using containerized services.
- Monitoring of deployed models for prediction latency, feature distribution, and drift detection.

1.3 Tools and Environment

- **Programming:** Python 3.10
- **Libraries:** Pandas, NumPy, Matplotlib, Scikit-learn, TensorFlow
- **MLOps:** MLflow, Evidently AI, Apache Airflow, Vercel
- **Automation:** Apache Airflow
- **Versioning:** GitHub, DVC
- **Deployment:** Docker, Vertex AI, Google Cloud Platform
- **Monitoring & Analytics:** BigQuery and Looker Studio

1.4 Literature Review

Sun Shuai Ling (2024) performed unsupervised clustering algorithms including centroid-based clustering (K-means), hierarchical clustering, Density-Based Spatial Clustering of Applications with Noise (DBSCAN clustering), and distribution-based clustering (Gaussian Mixture Model) with recency, frequency, monetary (RFM) modeling to enhance customer segmentation within a customer relationship management (CRM) system.

DOI: 10.1109/ISCAIE61308.2024.10576353 [IEEE](#)

Victor Hugo et al. (2024) studied transaction data obtained from one of the e-commerce stores, i.e. MurahJaya888 and followed by analysis using various clustering methods such as K-means, Kmedoids, Fuzzy c-means, and Mini-batch k-means. We also proposed a new model that will become the attributes cluster, namely, RFM + DP (Discount Proportion).

DOI: 10.14569/IJACSA.2024.0150390 [SAI](#)

A. J. Christy et al. (2021) performed segmentation using RFM, compared clustering approaches (incl. K-means/Fuzzy methods), and proposed improved initialization ideas. Benefited segmentation quality by stabilizing clusters and improving interpretability for marketing actions and gaining credibility over traditional models by incorporating key influencing factors. This approach demonstrates that combining statistical and deep learning methods can capture complex dependencies in e-commerce segmentation data.

[RFM Ranking – An Effective Approach to Customer Segmentation](#)

R. Vohra et al.. (2020) Combined SOM (topology-preserving mapping) with K-means on RFM features for online retail segmentation. Benefited segmentation by capturing nonlinear structure before partitioning customers into groups.

[Using Self-Organizing Maps and K-Means Clustering Based on RFM Model](#)

G. Aslantaş (2023) Adapted K-means around RFM-style behavioral features and demonstrated segment profiling. Benefited by producing clearer “high value / low value / recent / frequent” customer groups for targeted campaigns

[Customer Segmentation Using K-Means Clustering \(RFM-oriented\)](#)

D. A. Awaliyah et al. (2024) Focused on using K-means for online retail transaction segmentation, emphasizing preprocessing/variable selection for clustering. Benefited by showing practical steps to improve segmentation outcomes on retail transaction data.

[Optimizing Customer Segmentation in Online Retail Transactions](#)

Kiran Kumar Patibandla et al. (2025) Integrated PCA to reduce dimensionality and then applied K-means on retail-derived features (e.g., RFM). Benefited by improving cluster separability/visualization and reducing noise in segmentation.

[Enhancing Online Retail Insights: K-Means Clustering and PCA](#)

A. Wasilewski et al. (2024) Compared multiple clustering algorithms using multi-criteria decision methods under personalization constraints. Benefited by guiding algorithm choice beyond a single metric (useful when selecting K-means vs alternatives).

[Multi-factor Evaluation of Clustering Methods for E-commerce Personalization](#)

M. Wang et al. (2025) Used text/review semantics (BERT/LDA representations) with deep embedded clustering for segmentation. Benefited by moving beyond transactions into language-driven segmentation for richer customer intent insights.

[Deep Segmentation of Retail Customers Based on Improved Deep Embedded Clustering](#)

Z. Ling et al. (2025) Combined clustering with deep learning in a precision marketing context to improve segmentation-response linkage. Benefited by connecting segmentation outputs to marketing effectiveness (conversion/response modeling).

[Research on Consumer Segmentation](#)

M V Rajesh et al. (2024) illustrates the implementation of the marketing technique called RFM model along with the k-means and BIRCH machine learning clustering algorithms on the e-commerce customers sales dataset resulting in fruitful customer segmentation. A comparative analysis is also performed which resulted in k-means outperforming the BIRCH

DOI:10.2991/978-94-6463-471-6_94 [Atlantis](#)

Shuhai Wang et al. (2024) proposes a dynamic customer segmentation approach by combining LRFMS and multivariate time series clustering. Firstly, this method represents each customer

behavior as a time series sequence of the Length, Recency, Frequency, Monetary and Satisfaction variables. And then, we apply a multi-dimensional time series clustering algorithm based on three distance measurement methods called DTW-D, SBD, and CID to carry out customer segmentation. Finally, an empirical study and comparative analyses are conducted using customer transaction data of parts agents to verify the effectiveness of the approach. Additionally, a detailed analysis of different customer groups is made, and corresponding marketing suggestions are provided. DOI:10.1038/s41598-024-68621-2 [NIH](#)

Yohanni Syahra et al. (2025) implemented the RFM + K-means approach on a medium-size retail dataset and identifies segments like “Loyal,” “Potential Loyalists,” “At-Risk,” and “One-Time Buyers,” aiding targeted CRM strategies. DOI:10.33395/sinkron.v9i3.14974 [SNL](#)

Naresh Kumar (2025) Customer segmentation was carried out using Recency, Frequency, and Monetary (RFM) values derived from transaction records and grouped using the Bisecting K-Means algorithm. From an initial dataset of 541,909 entries, 397,884 valid records were retained after removing missing values, negative quantities, and duplicate rows. RFM values were calculated and normalized before clustering, and the Elbow method confirmed three as the optimal number of customer groups. The resulting clusters reflected clear behavioral patterns: one group showed high spending and frequent purchases, another included inactive users, and the third represented recent but low-spending customers. DOI:10.1007/s43995-025-00180-7 [SNL](#)

Firas Bayram et al. (2025) provide a comprehensive overview of the trustworthiness property of MLOps systems. Specifically, we highlight technical practices to achieve robust MLOps systems. In addition, we survey the existing research approaches that address the robustness aspects of ML systems in production. We also review the tools and software available to build MLOps systems and summarize their support to handle the robustness aspects. Finally, we present the open challenges and propose possible future directions and opportunities within this emerging field. The aim of this paper is to provide researchers and practitioners working on practical AI applications with a comprehensive view to adopt robust ML solutions in production environments. DOI: 10.1145/3708497 [ACM](#)

Dominic Kreuzberger et al. (2022) conduct mixed-method research, including a literature review, a tool review, and expert interviews. As a result of these investigations, we provide an aggregated overview of the necessary principles, components, and roles, as well as the associated

architecture and workflows. Furthermore, we furnish a definition of MLOps and highlight open challenges in the field. Finally, this work provides guidance for ML researchers and practitioners who want to automate and operate their ML products with a designated set of technologies. DOI:10.48550/arXiv.2205.02302 [arXiv](#)

Elamurugan Balasundaram(2024) adopted a hybrid approach, utilizing k-means clustering for customer segmentation based on recency, frequency, and monetary values, followed by an XGBoost classifier application for loyalty prediction. The methodology involved analyzing actual e-commerce data and comparing results with established industry benchmarks. The hybrid model demonstrated superior performance over conventional methods, evidenced by improved precision, recall, accuracy, and F1 scores in loyalty prediction, alongside higher silhouette scores and lower Davies–Bouldin indices for customer segmentation. DOI:10.17010/pijom/2024/v17i10/173996 [IJM](#)

The reviewed literature highlights a strong consensus that customer segmentation remains a foundational task in data-driven decision-making, with RFM-based feature engineering and clustering techniques continuing to be widely adopted due to their effectiveness and interpretability. Recent studies emphasize that while advanced methods such as deep embedded clustering, hybrid models, and ensemble approaches can enhance segmentation quality in complex or high-dimensional settings, K-Means clustering remains a robust and scalable baseline for large transactional datasets.

Across the literature, internal validation metrics such as the Silhouette Score, Davies–Bouldin Index, and Calinski–Harabasz Index are consistently recommended for evaluating unsupervised models. Additionally, contemporary research underscores the importance of operationalizing machine learning through **MLOps practices**, including automated pipelines, experiment tracking, version control, monitoring, and drift detection. These practices bridge the gap between experimental models and real-world deployment, ensuring long-term reliability and governance. Collectively, the literature supports an integrated approach that combines interpretable clustering techniques with strong MLOps frameworks, validating the design choices and methodology adopted in this project.

Chapter 2

Understanding MLOps

Machine Learning Operations (MLOps) integrates machine learning, software engineering, and DevOps practices to enable reliable deployment, monitoring, and maintenance of ML models in production. Unlike traditional software, ML systems are highly data-dependent and can degrade over time due to changing data distributions. MLOps addresses this by enabling automated workflows for model versioning, deployment, monitoring, and retraining.

In e-commerce segmentation, where market conditions and regulations frequently change, MLOps principles are applied to transform a customer segmentation model from a standalone analytical experiment into a production-grade, end-to-end machine learning system.

Risks in Machine Learning Systems

Data-Related Risks

- **Data Drift:** Changes in data distribution over time
- **Data Quality Issues:** Missing, noisy, or inconsistent data
- **Data Leakage:** Use of unintended future information

Model-Related Risks

- **Overfitting:** Poor generalization to unseen data
- **Concept Drift:** Changes in input–output relationships
- **Limited Interpretability:** Difficulty explaining model behavior

Operational Risks

- **Pipeline Failures:** Errors in data ingestion or scheduling.
- **Scalability Issues:** Inability to handle increased workloads
- **Environment Inconsistency:** Differences between development and production MLOps

mitigates these risks through automation, monitoring, and standardized environments.

Requirement Analysis

Functional Requirements

- Automated data ingestion and preprocessing
- Support for multiple forecasting models
- Systematic training, evaluation, and comparison
- Forecast storage and visualization
- Data drift detection and retraining triggers
- Full experiment and model version tracking

Non-Functional Requirements

- Scalability and reliability
- Reproducibility and maintainability
- Secure access to data and models
- Efficient use of computational resources

Risk Mitigation Using MLOps

Risk mitigation is achieved through:

- **Version Control:** Git and DVC
- **Experiment Tracking:** ML FLOW
- **Pipeline Automation:** Apache Airflow
- **Containerization:** Docker
- **Monitoring and Drift Detection:** Evidently AI

Risk Assessment Matrix

The risk assessment matrix is used to systematically identify, evaluate, and prioritize risks across the MLOps lifecycle based on their likelihood and potential impact. It highlights critical risks such as data drift, concept drift, and model degradation that can significantly affect model performance and decision-making. By categorizing risks into priority levels, the matrix supports proactive mitigation planning, accountability, and continuous monitoring. This structured approach ensures that potential issues are detected early and addressed effectively, contributing to the reliability, stability, and governance of the deployed machine learning system.

R1: Data Drift – Changes in input data distribution over time

R2: Concept Drift – Change in relationship between inputs and target variable

R3: Model Degradation – Decline in predictive accuracy over time

R4: Poor Data Quality – Missing, noisy, or inconsistent data

R5: Pipeline Failure – Failure in data ingestion or training pipeline

R6: Deployment Errors – Incorrect model version or configuration deployed

R7: Latency Issues – Increased inference response time

R8: Security & Access Risks – Unauthorized access to models or data

R9: Regulatory Non-Compliance – Failure to meet reporting or audit requirements

R10: Monitoring Gaps – Insufficient visibility into production model behavior

	Negligible	Minor	Moderate	Significant	Severe
Very Likely					R1 R3
Likely			R4 R7 R10		R2
Possible					
Unlikely				R6	R5 R8 R9
Very Unlikely					

Impact →

Figure 1 : Risk assessment matrix for customer spending trends in an MLOps framework

Chapter 3

Data Preprocessing

3.1 Data Collection and Exploration

The data used in this project was collected from a publicly available e-commerce transactional dataset containing detailed records of customer purchases over a one-year period. The dataset includes key attributes such as invoice numbers, product identifiers, purchase quantities, transaction timestamps, unit prices, customer identifiers, and geographic information. During the exploration phase, an initial assessment of data structure, size, and data types was conducted to understand the scope and quality of the dataset.

3.2 Data Profiling, Cleaning, and Preprocessing

Data profiling was performed to assess the completeness, consistency, and reliability of the collected dataset prior to model development. This involved analyzing missing values, identifying duplicate transactions, examining value ranges, and validating data types across all attributes. Based on the profiling results, a systematic data cleaning process was implemented, including the removal of duplicate records, handling of missing customer identifiers, exclusion of cancelled transactions, correction of anomalous product codes, and elimination of records with zero or invalid unit prices.

3.3 Feature Extraction and Selection

Feature extraction and selection were conducted to transform raw transactional data into meaningful representations of customer behavior suitable for clustering. Key behavioral features were derived using the Recency, Frequency, and Monetary (RFM) framework to capture purchasing regularity and customer value. Additional features such as average transaction value, total number of unique products purchased, cancellation frequency, time-of-day purchasing patterns, inter-purchase intervals, geographic indicators, and seasonal

spending trends were extracted to enrich customer profiles.

3.4 Model Design Approaches

The model design in this project is centered on an unsupervised learning approach to customer segmentation, with K-Means clustering selected as the primary algorithm due to its scalability, simplicity, and effectiveness for large transactional datasets. Prior to clustering, Principal Component Analysis (PCA) was employed to reduce dimensionality, mitigate multicollinearity, and enhance cluster separability while preserving the majority of data variance. The model design involved careful selection of the number of clusters using internal validation metrics such as the Silhouette Score, Davies–Bouldin Index, and Calinski–Harabasz Index. Multiple initialization strategies and iterations were tested to improve convergence stability and reduce sensitivity to centroid initialization. The clustering model was designed to integrate seamlessly into an automated MLOps pipeline, enabling repeatable training, evaluation, and deployment. This design approach balances model interpretability with computational efficiency, ensuring that the resulting customer segments are both actionable for business insights and suitable for scalable production deployment.

Chapter 4

Model Building and Evaluation

4.1 Model Building

The model building phase involved evaluating multiple machine learning approaches to identify the most effective method for customer segmentation. Several algorithms were implemented and compared, including K-Means clustering, DBSCAN, hierarchical clustering using dendrograms, and an ensemble-based approach combining dendrogram-based clustering with DBSCAN. In addition, a Random Forest-based model was explored to analyze feature importance and support behavioral pattern understanding.

Three models were implemented:

4.1.1 K-Means Clustering Model

K-Means delivered the best overall performance, producing well-separated and interpretable clusters with high silhouette scores and low intra-cluster variance. It scaled efficiently to large datasets and was selected as the final model.

4.1.2 Ensemble of Dendrogram and DBSCAN:

Hierarchical clustering provided clear insights into customer relationships but was computationally expensive and not scalable for large datasets. DBSCAN was sensitive to parameter selection and struggled with varying customer density, resulting in excessive noise points and poorly defined clusters. The ensemble approach offered slight robustness improvements over individual methods but did not significantly enhance cluster quality or stability.

4.1.3 Random Forest

Random Forest was used primarily for feature importance analysis rather than clustering and did not produce meaningful customer segments.

4.2 Training, Validation and Testing

To ensure fair and realistic evaluation, a structured training and evaluation strategy was adopted.

4.2.1 Training Strategy

Models were trained using standardized and preprocessed features to ensure consistency across approaches. Multiple initializations and iterations were used, particularly for K-Means, to improve stability and convergence.

4.2.2 Validation

Validation was performed using internal clustering metrics such as Silhouette Score, Davies–Bouldin Index, and Calinski–Harabasz Index to evaluate cluster cohesion, separation, and overall quality.

4.2.3 Testing

- As the problem is unsupervised, testing focused on cluster stability, interpretability, and consistency across multiple runs and data samples rather than labeled test sets.
- This ensured an unbiased assessment of each model’s performance on unseen e-commerce price data.

The training-validation-testing pipeline ensures model robustness and reflects real-world forecasting conditions.

4.3 Evaluation Metrics and Visualization

Model performance was assessed using standard regression metrics and visualization techniques.

4.3.1 Evaluation Metrics

- Silhouette Score: Used to measure how well data points fit within their assigned clusters compared to other clusters, with higher values indicating better separation.
- Davies–Bouldin Index: Evaluates cluster compactness and separation, where lower values represent better clustering performance.
- Calinski–Harabasz Index: Assesses the ratio of between-cluster dispersion to within-cluster dispersion, with higher scores indicating well-defined clusters.

These metrics enable consistent comparison across all models.

4.3.2 Visualization

Visualizations were employed to complement quantitative evaluation:

- Actual vs Predicted Price Plots to assess alignment between forecasts and real prices.
- Residual Error Plots to identify bias or systematic prediction errors.
- Time Series Forecast Graphs to visualize model behavior over different periods.

Visualization enhances interpretability and helps identify strengths and limitations of each model.

4.4 Experiment Tracking

To maintain reproducibility and transparency, all experiments were systematically tracked. Tracked information included:

- Model type and configuration
- Hyperparameter values
- Training, validation, and test metrics
- Performance comparison across model iterations
- System memory and network metrics

Experiment tracking facilitates structured experimentation, enabling efficient comparison of models and selection of the best-performing approach for e-commerce price forecasting.



Fig 1: System Metrics Evaluation and Monitoring in ML FLOW

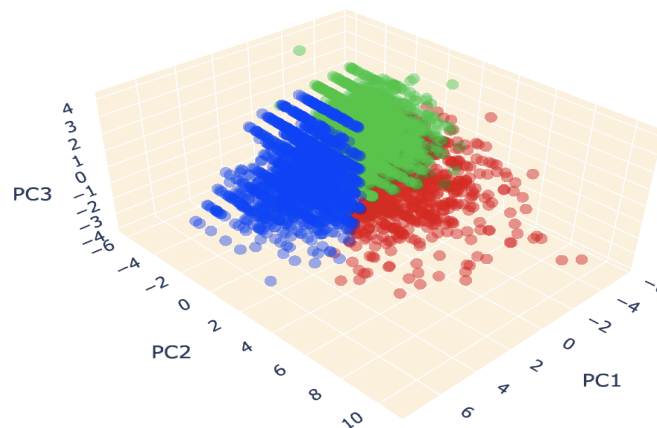


Fig 2: 3D Visualization of Customer Clusters in PCA Space

4.5 Interpreting Model Results and Error Analysis

4.5.1 Model Result Interpretation

Each model demonstrated distinct performance characteristics:

- **K-Means Clustering:** Performed the best among all models by producing clear, well-separated, and stable clusters. It handled the large dataset efficiently and delivered high silhouette scores.
- **Ensemble of Dendrogram and DBSCAN:** Showed slight improvement over individual methods but did not significantly enhance cluster quality or consistency, behavior, producing smoother forecasts.
- **Random Forest** Was mainly used for understanding feature importance and did not directly produce meaningful customer clusters.

4.5.2 Error Analysis

Error analysis revealed that:

- Prediction errors were higher during periods of high volatility.
- Time series models were sensitive to abrupt market shifts.
- Machine learning models benefited from richer feature representations but required careful tuning.

Chapter 5

Images

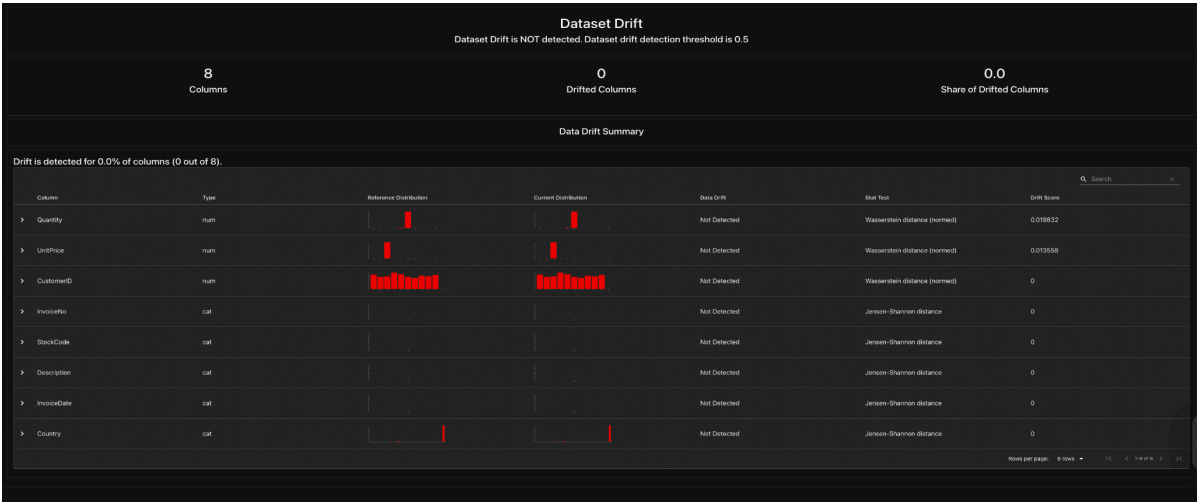


Fig 3 : Data Drift detection through distribution

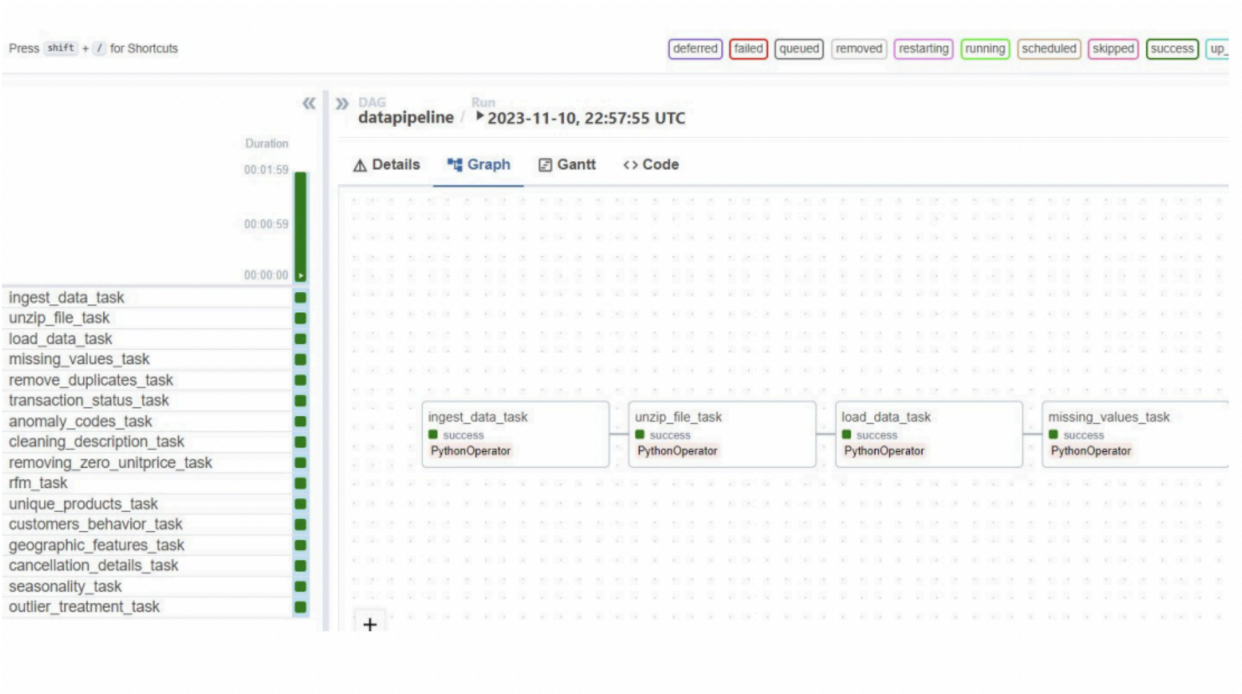


Fig 4 : Apache Airflow Automation Process

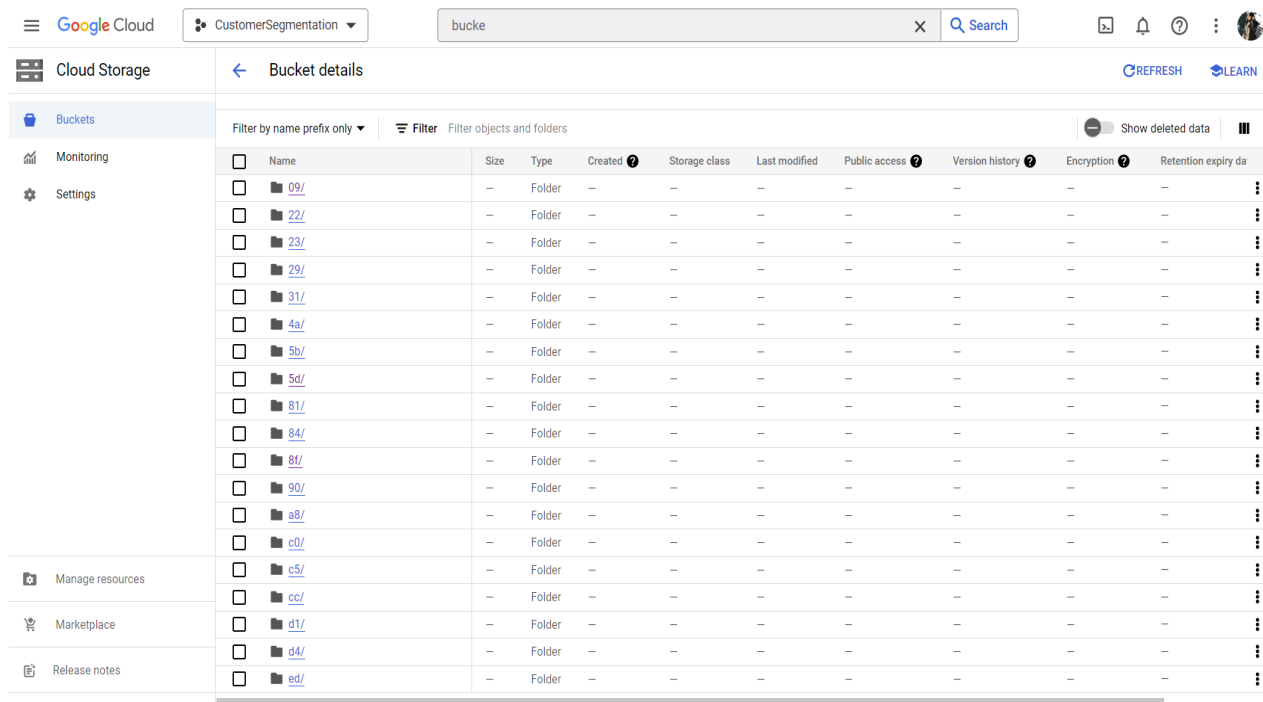


Fig 5 : Data and model versioning using DVC

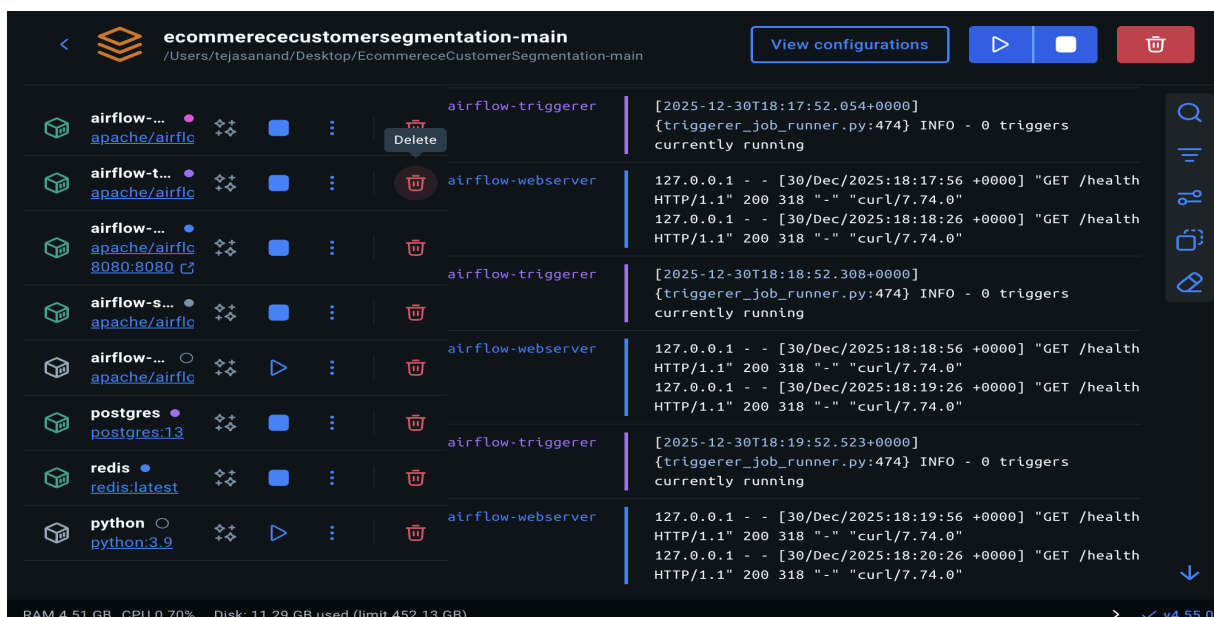


Fig 5 : Docker Containerization

Chapter 6

Deployment, Monitoring and Governance

This chapter discusses the activities carried out after model development to ensure that the e-commerce segmentation model remains reliable, scalable, and well-governed in a real-world environment. The focus is on continuous performance evaluation, controlled optimization, interpretability, version management, and systematic documentation throughout the model lifecycle.

6.1 Revisiting Model Performance

After deployment-ready models were identified, their performance was periodically revisited using newly available e-commerce trading data. Forecasts were generated in a rolling-window fashion to simulate real-time prediction scenarios.

Model evaluation metrics such as Silhouette Score and others were automatically logged across successive runs using experiment tracking platforms such as **MLflow**. This platform enabled continuous comparison between historical and recent runs, making it possible to detect performance drift arising from changes in market behavior.

Visualization dashboards provided by ML Flow further supported performance monitoring by highlighting trends in prediction error over time. This continuous feedback loop ensured that degradation in model accuracy was identified early.

6.2 Hyperparameter Tuning and Performance

Hyperparameter tuning was performed to optimize the performance and stability of the clustering models evaluated in this project. For K-Means clustering, key hyperparameters such as the number of clusters (k), initialization method, number of initializations (n_{init}), and maximum iterations were systematically tuned. Different values of k were tested, and optimal cluster selection was guided by internal validation metrics including the Silhouette Score, Davies–Bouldin Index, and Calinski–Harabasz Index. Increasing the number of initializations improved convergence stability and reduced sensitivity to centroid initialization.

DBSCAN hyperparameters, including epsilon (ϵ) and minimum samples, were explored but resulted in inconsistent clustering due to varying data density. Hierarchical clustering linkage methods were also tested, though scalability limitations affected performance. Overall, **K-Means achieved the best balance between cluster quality, stability, and computational efficiency**, making it the final model choice for deployment.

6.3 Interpretability

Interpretability plays a crucial role in ensuring that the results of machine learning models are understandable, trustworthy, and actionable for stakeholders. In this project, interpretability is emphasized by selecting K-Means clustering, an inherently interpretable unsupervised learning algorithm. The centroid-based nature of K-Means allows each customer segment to be explained through average feature values, making it easier to understand the behavioral characteristics that define each cluster. This transparency supports informed business decision-making and increases confidence in the model outputs.

To further enhance interpretability, clusters were analyzed using key behavioral features such as recency, frequency, monetary value, transaction timing, and cancellation patterns. By examining centroid values and feature distributions within each cluster, meaningful customer profiles were derived, such as high-value frequent buyers or low-engagement occasional shoppers.

Visualization techniques, including PCA-based plots and cluster distribution charts, were used to provide intuitive representations of segmentation results, enabling non-technical stakeholders to easily grasp customer behavior patterns.

6.4 Model Versioning and Registry

To ensure reproducibility and controlled deployment, a structured model versioning strategy was implemented. Trained models were registered and versioned using **Docker** and also logged via **DVC**, with each version associated with its parameters, performance metrics, and training context.

Versioned models were containerized using **Docker**, ensuring consistency across development, testing, and deployment environments. Code and configuration changes were managed using **GitHub**, enabling traceability and rollback when required.

This governance framework ensured that only validated models were promoted for use while preserving the ability to revert to earlier versions if necessary.

6.5 Documentation of Model Updates

Thorough documentation was maintained throughout the deployment lifecycle to support transparency and long-term maintainability. Experiment histories recorded by ML Flow and **Google Cloud** provided an automated record of model evolution, while supplementary documentation was maintained through project reports and repository-level documentation. Automated workflows orchestrated using **Apache Airflow** were documented to describe scheduled retraining, evaluation, and logging processes. Each model update included a clear description of the motivation for change, modifications applied, and comparative performance outcomes. This approach ensured accountability, facilitated collaboration, and supported governance and audit requirements.

Chapter 7

CI/CD for ML (Automation & Orchestration)

This chapter details the automation of the e-commerce price forecasting lifecycle, ensuring that code, data, and models are integrated and delivered through a robust pipeline.

7.1 Overview of CI/CD for ML

Standard CI/CD focuses on code reliability. In this project, we implemented a Continuous Integration (CI) for code and data, and Continuous Deployment (CD) for the model service. The pipeline ensures that every update to the e-commerce customer price prediction logic is automatically tested against historical data versions (tracked via DVC) before being deployed to Render.

7.2 Tools Setup: GitHub Actions

The automation engine for this project is GitHub Actions.

The CI/CD pipeline in this project was established to automate code integration, testing, and deployment of machine learning components. **GitHub Actions** serves as the primary CI/CD tool, triggering automated workflows on every code push and pull request. The pipeline includes stages for code linting, unit testing, data pipeline validation, and model-related checks to ensure consistency and reliability. Upon successful validation, Docker images are automatically built and prepared for deployment. This CI/CD setup enforces code quality standards, reduces manual errors, and enables rapid, repeatable updates to the machine learning pipeline, supporting robust and scalable MLOps practices.

7.3 Automating Model Training and Testing

The project employs a dual-layered approach to automation:

- **Workflow Orchestration (Apache Airflow):** Airflow manages the Directed Acyclic Graph (DAG) for training. It schedules tasks such as `fetch_data`, `preprocess_data`, and `train_models`. If the `fetch_data` task fails (e.g., due to an API timeout), the pipeline prevents the training of stale models.
- **Unit & Integration Testing:** Before deployment, the pipeline runs automated tests to verify:
 - **Data Integrity:** Ensuring no null values exist in any column.
 - **Model Output:** Verifying that the model returns a valid float for future dates.
 - **Performance Gate:** Ensuring the new model's RMSE does not exceed a predefined baseline.

7.4 Building and Running a Deployment Pipeline

The deployment pipeline was designed to automate the transition of the trained customer segmentation model from development to production. The finalized K-Means model was containerized using **Docker**, ensuring consistency across environments. A **CI/CD pipeline** implemented through GitHub Actions automated model validation, testing, and image building upon successful code commits. The containerized model was then deployed on **Google Cloud Vertex AI**, where it was exposed as a scalable inference endpoint using a **Flask-based REST API**. The pipeline supports repeatable deployments, version control, and rollback mechanisms, enabling seamless model updates. Once deployed, the pipeline continuously logs prediction inputs, outputs, and latency metrics, allowing the system to run reliably in a production environment while supporting monitoring and future retraining workflows.

Chapter 8:

Model Monitoring and Retraining

Post-deployment, the system must handle "Model Decay" caused by the volatile nature of the e-commerce market.

8.1 Model Monitoring Concepts and KPIs

Model monitoring is a critical component of the MLOps lifecycle, ensuring that the deployed customer segmentation model continues to perform reliably in production. Monitoring focuses on tracking model behavior, data quality, and system performance over time to detect issues such as data drift, concept drift, and model degradation. By continuously observing key indicators, the system enables early detection of anomalies that could impact segmentation accuracy and business decisions.

8.2 Monitoring Tools: Google BigQuery and Looker Studio

To ensure reliable operation of the deployed model, a set of monitoring tools was integrated into the MLOps pipeline. **Google BigQuery** is used to store prediction logs, input feature values, cluster assignments, and inference timestamps, enabling large-scale analysis of model behavior. **Looker Studio** is utilized to create interactive dashboards that visualize key metrics such as prediction latency, cluster distribution trends, and feature value ranges, supporting real-time observability. **Vertex AI monitoring capabilities** assist in tracking endpoint health, request volume, and system performance. Together, these tools provide end-to-end visibility into model performance, data drift, and operational stability, enabling proactive detection of issues and continuous model improvement.

8.3 Detecting Concept Drift and Data Drift

Detecting data drift and concept drift is essential to maintaining the reliability of machine learning models deployed in production. **Data drift** occurs when the statistical distribution of input features changes over time compared to the training data, while **concept drift** arises when the relationship between input features and model outputs changes, leading to reduced model effectiveness. In this project, data drift is monitored by continuously tracking feature distributions such as purchase frequency, monetary value, and transaction timing, and comparing them against baseline training distributions using statistical summaries and threshold-based alerts. **Concept drift** is identified by observing shifts in cluster assignments, changes in cluster proportions, and declining stability in customer segmentation patterns over time. Logging and analysis of live prediction data enable early detection of these drifts, triggering model review or retraining when predefined thresholds are exceeded. This proactive drift detection strategy ensures sustained model performance and supports timely adaptation to evolving customer behavior.

8.4 Model Retraining and Redeployment

Model retraining and redeployment are essential to ensure that the deployed machine learning model remains accurate and relevant as data patterns evolve. In this project, retraining is triggered by indicators such as data drift, concept drift, or a decline in clustering stability observed through continuous monitoring. Updated datasets are versioned using **DVC**, and new training runs are tracked with **MLflow** to compare performance against existing production models. Once a retrained model demonstrates improved validation metrics and cluster stability, it is containerized and redeployed through the automated **CI/CD pipeline**. The redeployment process includes version control, validation checks, and rollback mechanisms to minimize operational risk. This structured retraining and redeployment strategy ensures continuous improvement, model reliability, and seamless adaptation to changing customer behavior in production environments.

8.5 Logging, Alerting, and Dashboard Setup

Logging, alerting, and dashboarding are integral components of the monitoring framework used to ensure the reliability and transparency of the deployed machine learning model. In this project, detailed logs capturing input features, predicted cluster labels, inference timestamps, and latency metrics are stored in **Google BigQuery** for centralized analysis and traceability. Alerting mechanisms are configured to trigger notifications when predefined thresholds are exceeded, such as abnormal increases in latency, significant shifts in feature distributions, or sudden changes in cluster assignment patterns. **Looker Studio dashboards** provide real-time visualization of key performance indicators, including system availability, prediction trends, and drift indicators. This integrated setup enables proactive issue detection, supports rapid response, and facilitates continuous performance monitoring and decision-making throughout the model's lifecycle.

Chapter 9:

Post-Deployment Governance and Continuous Improvement

Post-deployment governance ensures that the deployed model remains reliable, compliant, and aligned with business objectives over time. Continuous monitoring, version control, and audit mechanisms are used to track model performance, data quality, and ethical compliance. Feedback from monitoring and periodic reviews informs retraining, model updates, and pipeline improvements. This structured governance approach supports continuous improvement, minimizes operational risks, and ensures long-term sustainability of the machine learning system.

9.1 Post-Deployment Performance Review

- Periodic evaluation using new customer spending trends trading data
- Comparison with baseline model performance
- Logging of metrics using MLflow and Vertex AI
- Monitoring of:
 - Silhouette Score
 - Davies–Bouldin Index
 - Calinski–Harabasz Index

9.2 Continuous Improvement via Feedback Loops

- Automated feedback loops enabled using MLOps tools
- Drift detection reports generated by Evidently AI
- Retraining triggered by:
 - Performance degradation
 - Drift alerts
- Apache Airflow manages retraining workflows

9.3 Governance and Ethical AI Practices

- Ensures transparency and responsible use of forecasts
- Reproducibility maintained using GitHub and DVC
- ML FLOW is also used for this purpose

9.4 Documentation: Model Cards and Data Cards

- Model Cards document model purpose, metrics, and limitations
- Data Cards describe dataset source, preprocessing, and usage constraints
- Use to give a clear picture of the system

9.5 Audit and Review Checklist

- **Technical Audit:**
 - Verification of pipeline reproducibility across environments
 - Validation of code consistency, version control, and CI/CD compliance
 - Review of automated testing and dependency management

- **Data Audit:**
 - Data integrity checks for completeness, accuracy, and consistency
 - Verification of data versioning and lineage
 - Monitoring and validation of data drift indicators

- **Model Audit:**
 - Review of clustering performance stability using evaluation metrics
 - Validation of model version consistency and traceability
 - Assessment of cluster interpretability and stability over time

- **Deployment Audit:**
 - Verification of container integrity and configuration
 - Review of cloud infrastructure reliability and scalability
 - Validation of rollback and recovery mechanisms

- **Ethical Audit:**
 - Assessment of transparency and documentation completeness
 - Verification of responsible AI practices and bias mitigation
 - Compliance with data privacy, security, and governance standards

Chapter 10:

End-to-End MLOps Pipeline Demonstration and Final Evaluation

This chapter presents the complete lifecycle demonstration and final evaluation of the e-commerce price forecasting system.

10.1 End-to-End MLOps Pipeline Demonstration

Demonstration of the full MLOps workflow:

- **Data ingestion and preprocessing**
- **Data versioning using DVC**
- **Model training and evaluation**
- **Experiment tracking with MLflow,**
- **Monitoring and drift detection**
- **Automated retraining and redeployment**

Apache Airflow orchestrates all pipeline stages using DAGs

Docker ensures environment consistency across development and production

GitHub Actions automate testing and deployment to Render

10.2 Final Model Evaluation and Results

- **Final evaluation performed on unseen test data**
- **Models evaluated:**
 - K Means
 - Ensemble Model
 - Random Forest
- **Evaluation metrics include:**
 - Silhouette Score
 - Davies–Bouldin Index
 - Calinski–Harabasz Index e
- **Observations:**
 - K Means had the best performance
 - Ensemble Model captures seasonality effectively
 - Random Forest handles non-linear patterns well

Final model selected based on accuracy, stability, and computational cost was K Means

10.3 Project Documentation and Reporting

Complete documentation maintained throughout the project

- GitHub repository includes:
 - Airflow DAGs
 - Docker configuration
 - Source code
- DVC tracks:
 - Dataset versions
 - Trained model artifacts
- Experiment logs provide full traceability of model evolution

10.4 Final Presentation and Evaluation

Final presentation structured for academic and technical evaluation

Key presentation points:

- Problem statement and motivation
- MLOps architecture
- Monitoring and retraining strategy
- Final results

Evaluation based on:

- Correct application of MLOps principles
- Deployment readiness
- Governance and ethical practices